ELI Security Posture

Contents

ELI runs locally with real OS reach (shell, file, app control) plus a user-extensible agent/plugin system — so the security model matters. It is **fail-closed** by design. Spans `eli/runtime/security.py`, the executor gate, the engine input sanitiser, the memory SQL validator, and the agent trust registry.

1. Shell command gate (`runtime/security.py SecurityManager`)

- `is_command_allowed(cmd)`:
  - `ELI_FULL_CONTROL=1` → allow all.
  - `ELI_ALLOWED_CMDS` contains * → allow all.
  - `ELI_ALLOWED_CMDS` unset **and** no full-control → **blocked (fail-closed)**; logs a warning explaining how to opt in. (Always allows the project's own `.venv/bin/python`.)
- `safe_subprocess(cmd, timeout)` — validates `cmd[0]` against the allowlist before running; capture_output, timeout-bounded, never `shell=True`.
- **Defence in depth:** `executor_enhanced._shell_command_allowed_fallback` mirrors this exact logic, so if `SecurityManager` fails to import, the executor still fails **closed** (never fail-open). Explicit comment to that effect.

2. Filesystem & app gates (`SecurityManager`)

- `is_path_allowed(path)` — must resolve within allow-roots (`project_root` + `$HOME` + `ELI_ALLOW_ROOTS`). Uses `Path.resolve()` + `relative_to`, so `..` traversal can't escape.
- `is_app_allowed(app)` — explicit `ELI_ALLOWED_APPS`, else a curated default-safe set (settings, file manager, editor, calculator, terminal, browser, mail). `ELI_FULL_CONTROL` bypasses.

3. Prompt-injection guard (`kernel/engine.py`)

`_ELI_INJECTION_PATTERNS` (regex) + `_eli_sanitize_user_input`: - Matches classic jailbreak/role-override prefixes — `### system:`, "ignore/disregard/forget (all) previous instructions", "you are now a dan/jailbreak/unrestricted/free" — and replaces the matched span with `[filtered]` (keeps the rest of the message). - Strips control characters. - Runs before the input reaches the LLM, so injected role-overrides are neutralised but legitimate content survives.

4. SQL identifier validation (`memory/memory.py`)

`_IDENTIFIER_RE = ^[A-Za-z_][A-Za-z0-9_]*$` + `_validate_identifier(name)` — called before **any** f-string interpolation of a table/column name into SQL (`_validate_identifier(table, "table")`, column DDL first-token check). Raises `ValueError` on anything non-conforming. Closes the one place dynamic SQL is unavoidable (parameterised values are used everywhere else).

5. Custom-agent trust registry (`cognition/agent_bus.py`)

Custom agents (GUI wizard / `$ELI_CUSTOM_AGENTS_DIR`) are **SHA-256 hash-gated** against `config/trusted_agents.json`. Unknown or modified files are skipped with a SECURITY debug line; approve via `eli --trust-agent <path>`. `ELI_TRUST_ALL_AGENTS=1` bypasses for dev only. Fail-closed (missing/mismatched hash ⇒ not loaded). See `orchestration_and_agents.md`.

6. Code generation & self-modification — three paths

ELI has **three** distinct codegen paths, with escalating capability and matching safeguards:

1. **Pre-vetted canned library** (`runtime/generated_script_guard.py`, 1.1k LOC) — ~5 hand-written, reviewed scripts (`_write_gpu_memory_watch_script`, `_write_type_ia_redshift_script`, `_write_ton_618_mass_density_script`, …) that ELI *writes to disk* on request. No LLM code is executed; safe but fixed to the curated set (several physics-domain).
2. **`GENERATE_SCRIPT`** (`execution/executor_enhanced.py`) — *real* LLM codegen. Detects language/intent, prompts for frontier-quality code, then gates it:
   - `_verify_python_module_apis` — imports the referenced libraries and confirms `module.attr` references actually exist (catches hallucinated APIs).
   - `_quality_reject_reason` — rejects refusals, stubs, too-short, no-real-compute, missing-plot, `required=True`-without-default, SyntaxError.
   - **`_sandbox_run_python` (added)** — executes the candidate in a bounded, isolated subprocess (temp cwd, scrubbed env, `MPLBACKEND=Agg` so `plt.show()` can't block, wall-clock timeout `ELI_GENSCRIPT_RUN_TIMEOUT=20s`, generous `RLIMIT_CPU`; **no `RLIMIT_AS`** — it breaks numpy/scipy). A genuine unhandled traceback is fed back as repair feedback into a **3-attempt regenerate loop**; timeouts / signal kills / missing-optional-deps are tolerated (never reject legit heavy scientific scripts). Disable with `ELI_GENSCRIPT_VERIFY_RUN=0`.
3. **Self-patching** (`runtime/self_improvement.py` `SelfImprovementEngine`) — ELI modifies its *own* source for recurring failures. Gated behind `auto_patch_enabled` (**default off**), ≤2 patches/cycle. Safeguards: verbatim-old-match → `ast.parse` → timestamped backup (+ canonical `.eli_bak`) → write → `py_compile` → **differential import smoke-test (added)**: the patched module is imported in an isolated subprocess and **auto-reverted** if it no longer loads, but only when the module imported cleanly *before* the patch (so missing optional deps don't cause false reverts). Project-root-confined, `.py`-only, logged to the `code_patches` table.

   Code-health flag: `generated_script_guard` uses the same stacked-override pattern as the grounding gate — two `install()` definitions chained via `_ELI_SQLITE_SCRIPT_PREVIOUS_INSTALL`. Works, but layered rather than edited.

   Possible next step (not done): the self-patch loop verifies the module still *imports*; it does not yet re-run the original failing input to confirm the patch actually *fixes* the failure. A behavioural/test re-run would close that last gap.

Honest assessment

- **Strong:** the gates are real and **fail-closed** — shell blocked by default, fallback mirror prevents fail-open, path traversal contained, identifiers validated, agents hash-trusted, injection prefixes stripped, and "generated" code is actually pre-reviewed. This is well above the norm for local-AI projects, which routinely ship wide-open shell access.
- **Weak / watch:**

1. The injection guard is **pattern-based** — it catches known prefixes, not novel/obfuscated injections (no model-side defence). Reasonable for a single-user local tool, insufficient if ever multi-tenant.
2. `ELI_FULL_CONTROL=1` is a single env var that disables *all* gates at once — convenient, but a blunt instrument; there's no middle "allow file but not shell" tier beyond the per-axis env vars.
3. ~~The default-safe app/command sets are Linux/GNOME-flavoured.~~ **RESOLVED** — the default-safe **app** set in `_is_default_safe_app` is cross-platform (Linux + macOS Finder/Safari/Terminal.app/Syst Settings + Windows Explorer/Notepad/cmd/PowerShell/ Control Panel). The **command** path is fail-closed (no OS-specific default set — blocked unless `ELI_ALLOWED_CMDS`/Full-Control), so nothing OS-specific to add there.
4. Pre-vetted-scripts approach is safe but doesn't scale — genuinely dynamic capability generation would need a real sandbox (resource limits, seccomp, subprocess isolation), which doesn't exist yet.

Update — 2026-06-09 (RUN_CMD terminal is real; mock leak fenced)

- **RUN_CMD uses a real `subprocess.run`** (`executor_enhanced.py:5350` — no shell, capture_output, timeout) behind the destructive-command security gate (`_BLOCKED_PATTERNS`: `rm -rf /`, `mkfs`, `dd of=/dev/`, `chmod 777 /`, fork bomb, shutdown/reboot). The gate is lifted only when **Full Control** is on. There is **no production mock path**.
- **The `<MagicMock …>` failure rows were test leakage, not runtime.** They came from `tests/test_shell_security_gate.py` patching `subprocess.run`; the executor's `(p.stdout or "") + (p.stderr or "")` concat then yields a MagicMock repr, which slipped into the live `agent.sqlite3` via a test-isolation gap. A guard in `SelfImprovementEngine.log_failure` now **drops any error/input carrying a Mock/MagicMock repr** at the write source, so a test isolation slip can no longer pollute real failures.

Web app & multi-user security (2026-06-28)

ELI now ships a self-hosted FastAPI web app (`api/server.py`). It is local-first and inherits all of the gates above (the model is the same local GGUF behind netguard; commands/files/apps stay fail-closed). The web surface adds:

7. Authenticated identity + RBAC (`eli/runtime/api_users.py`, `api/server.py`)

- **Three roles** — `admin` / `member` / `viewer` (read-only). Endpoints are dependency-gated (`require_admin` / `require_member` / `require_viewer` / `_require_token`). RBAC enforces once at least one user exists; before that, same-machine use is frictionless.
- **Fail-closed auth gate** — unauthenticated/under-privileged calls are rejected, not silently downgraded. The token store holds **hashes**, never raw tokens.

8. Tamper-evident, HMAC-keyed audit trail (`eli/runtime/evidence_ledger.py`)

- Every API action is recorded into a **hash-chained** ledger (who / what / outcome, **metadata only — never message content**). Any edited/deleted/reordered row is detected by `verify_chain()` and surfaced in the Audit tab + admin console.
- The chain is **HMAC-keyed** (`$ELI_AUDIT_HMAC_KEY`, else a `0600` key file beside the config), so the integrity check can't be forged by recomputing plain hashes.

9. Secret files born locked — TOCTOU closed (`eli/core/secure_io.py`)

- The old `write_text(...)` then `chmod(0o600)` pattern left a brief window where, on a typical umask, the file was born `0644` (world-readable) before the chmod landed — a real (if narrow) TOCTOU on a multi-user box, on exactly the files that matter most.

- **`secure_io.secure_write_text/bytes`** is now the single owner of secret writes: `tempfile.mkstemp` (born `0600`) → write → fsync → `os.replace` (atomic). The destination **never exists world-readable for any instant**, and readers see old-or-new, never partial.
- Routed through it: the **audit HMAC key**, the **API token store**, **settings** (may hold broker passwords/tokens), and the **agent-trust registry** (`trusted_agents.json`, the integrity anchor for §5 — hashes not secrets, but written `0600`/atomic so it can't be tampered or half-written).

10. Sandboxed research ingest

- Corpus ingest for the research workspace runs sandboxed (path-scoped), so a shared/ collaborative corpus can't reach outside its workspace.

11. Monitored Internet toggle + egress oversight (`eli/core/netguard.py` + `api/server.py`)

- ELI is **offline-by-default** and hard-gated at the **socket boundary** — the process-wide failsafe raises `OfflineError` on any non-loopback connect while the toggle is off, even from code that forgot the helpers (fail-closed; loopback/LAN-registered services allowed).
- The guard covers **all four** ways an outbound connection actually happens, cross-platform: `socket.socket.connect` (raises) and `socket.create_connection` (raises) — sync + the selector asyncio loop / urllib / requests; `socket.socket.connect_ex` — the **non-raising sibling** used by probes and health-checks, which is made to **return `ENETUNREACH`** when blocked (no socket is opened) so it can't silently bypass the block or the recorder; and a **best-effort patch of the Windows `ProactorEventLoop`** (`IocpProactor.connect`, overlapped `ConnectEx`), which never touches `socket.socket.connect`. The allowed path of each records egress identically (below); the Proactor patch is a no-op on non-Windows.
- The owner can deliberately **enable internet**: from the desktop GUI (Net toggle) or the web dashboard (`GET /v1/net` token-gated read + `POST /v1/net` **admin-only**), persisted via `network_enabled`. **Both flips are written to the audit ledger** (`net_toggle`, warning-severity on enable) — the web *and* the desktop toggle.
- **Egress is genuinely monitored, not a blind hole.** While network is on, the same socket chokepoint that enforces offline mode also **records every allowed non-loopback connection** (host:port + timestamp): into an **in-memory ring** (live tail; `GET /v1/net/egress` + the Overview widget) and, throttled to one row per host:port per window (`ELI_EGRESS_LOG_WINDOW`, default 300s), into the **tamper-evident audit ledger** as `net_egress` events. The ledger write is best-effort and runs on a background thread, so monitoring never delays or breaks a connection. Disable the ledger leg (keep the live ring) with `ELI_EGRESS_LEDGER=0`.
- The toggle changes *policy*; the socket failsafe still governs every actual connection, and every connection it allows is now on the record.
- Rationale: internet is "the final frontier to make ELI's world bigger" — available, but monitored (per-connection) and under the user's control, not a permanent lockout.

Honest assessment — web tier

- **Strong:** fail-closed auth, role separation, a tamper-evident HMAC'd audit trail, born-locked secret files, and a monitored (not absent) network path. For a self-hosted local AI this is well above the norm.
- **Watch:** RBAC is token-based and single-tenant-shaped; the injection guard (§3) is still pattern-based; `ELI_FULL_CONTROL` remains a blunt all-gates-off switch. None new, but they matter more once the web app is exposed beyond the owner's machine — keep it bound to trusted networks.

Update — 2026-07-02 (stable phone token, admin-gated model switch)

Two web-tier changes this cycle, both verified against the code:

- **Stable LAN token + rotate.** The phone's bearer token now **persists** across server restarts (`api/api_token.py`: env → a `0600` file under the config dir → generate-and-save), so a paired phone is no longer stranded every time the server bounces. A **rotate** button issues a fresh token and

invalidates the old one in one tap — the manual override for a lost/compromised phone. `/health` stays tokenless by design.

- **Model switching is admin-gated and allowlist-bound.** The dashboard's model dropdown posts to `POST /v1/model`, which requires an **admin** principal and only accepts a path already in the installed-models list (`GET /v1/models/installed`). A dropdown of real files can't strand the runtime on a bad path, and a non-admin can't switch the model. (The list endpoint was moved off `/v1/models` to `/v1/models/installed` so it no longer collides with the OpenAI-compatible route.)

Neither weakens the existing posture — the socket failsafe, fail-closed command gate, and audit ledger all still apply.